

BooleanExpression

BooleanExpression 사용법

QueryDSL에서 조건을 메서드로 분리하는 핵심 기법.
동적 쿼리의 기본 빌딩 블록이며, 코드 재사용성과 가독성을 크게 높인다.

왜 BooleanExpression을 쓰는가

Before: 조건을 where에 직접 작성

```
// 조건이 늘어날수록 where 절이 길어진다
queryFactory
    .selectFrom(oe)
    .where(
        orgName != null ? oe.orgName.contains(orgName) : null,
        type != null ? oe.orgType.eq(type) : null,
        activate != null ? oe.orgActivate.eq(activate) : null,
        startDate != null ? oe.createdAt.goe(startDate.atStartOfDay()) :
null
    )
    .fetch();
```

문제점:

- where 절이 비대해진다
- 같은 조건을 다른 쿼리에서 재사용할 수 없다
- 복잡한 조건 로직이 섞이면 읽기 어렵다

After: BooleanExpression으로 메서드 분리

```
// where 절이 깔끔해지고, 각 조건의 의미가 명확하다
queryFactory
    .selectFrom(oe)
    .where(
        orgNameContains(orgName),
        orgTypeEq(type),
        orgActivateEq(activate),
        createdAfter(startDate)
    )
    .fetch();
```

1. 기본 사용법

BooleanExpression은 **where** 조건 하나를 반환하는 메서드다.

```
import com.querydsl.core.types.dsl.BooleanExpression;

// 값이 있으면 조건 반환, 없으면 null 반환
private BooleanExpression orgNameContains(String orgName) {
    return StringUtils.hasText(orgName) ? oe.orgName.contains(orgName) :
    null;
}

private BooleanExpression orgTypeEq(OrganizationType type) {
    return type != null ? oe.orgType.eq(type) : null;
}
```

핵심: null을 반환하면 where에서 자동으로 무시된다

```
.where(
    orgNameContains(null),           // → null → 조건에서 제외
    orgTypeEq(OrganizationType.TEAM), // → 조건 적용
    orgActivateEq(null)             // → null → 조건에서 제외
)
// 실제 실행: WHERE oe.org_type = 'TEAM'
```

2. 이점

가독성

```
// 무슨 조건인지 한눈에 안 보임
.where(
    keyword != null && !keyword.isBlank()
        ? qi.mbName.contains(keyword).or(qo.orgName.contains(keyword))
        : null,
    position != null ? qa.mbPosition.eq(position) : null
)

// 메서드 이름만 봐도 의미가 명확
.where(
    keywordContains(keyword),
    positionEq(position)
)
```

재사용

```
// 같은 조건을 여러 쿼리에서 재사용 가능
// 데이터 조회 쿼리
queryFactory.selectFrom(oe)
    .where(orgNameContains(name), orgTypeEq(type))
    .fetch();

// count 쿼리에서도 동일하게 사용
queryFactory.select(oe.count()).from(oe)
    .where(orgNameContains(name), orgTypeEq(type))
    .fetchOne();
```

테스트 용이성

```
// 조건 메서드를 개별적으로 테스트할 수 있다
@Test
void orgNameContainsReturnsNullWhenEmpty() {
    assertThat(orgNameContains("")).isNull();
    assertThat(orgNameContains(null)).isNull();
}

@Test
void orgNameContainsReturnsExpressionWhenHasValue() {
    assertThat(orgNameContains("보건")).isNotNull();
}
```

조합 가능

```
// BooleanExpression끼리 .and() / .or()로 조합 가능
private BooleanExpression isActiveDepartment() {
    return orgTypeEq(OrganizationType.DEPARTMENT)
        .and(orgActivateEq(true));
}
```

3. 프로젝트 실제 사용 패턴

패턴 A: 단순 일치 조건

```
// OrganizationSearchRepoImpl.java
private BooleanExpression orgTypeEq(OrganizationType type) {
    return type != null ? oe.orgType.eq(type) : null;
}
```

패턴 B: 문자열 부분 검색

```
// OrganizationSearchRepoImpl.java
private BooleanExpression orgNameContains(String orgName) {
    return StringUtils.hasText(orgName) ? oe.orgName.contains(orgName) :
    null;
}
```

패턴 C: 여러 컬럼 OR 검색

```
// ScheduleSearchRepoImpl.java
private BooleanExpression keywordContains(String keyword) {
    if (!StringUtils.hasText(keyword)) return null;
    return se.title.contains(keyword)
        .or(se.location.contains(keyword))
        .or(se.content.contains(keyword));
}
```

패턴 D: 날짜 범위

```
// ScheduleSearchRepoImpl.java
private BooleanExpression startFromGoe(LocalDate startFrom) {
    return startFrom != null ? se.startAt.goe(startFrom.atStartOfDay()) :
    null;
}

private BooleanExpression endToLoe(LocalDate endTo) {
    return endTo != null ? se.endAt.lt(endTo.plusDays(1).atStartOfDay()) :
    null;
}
```

패턴 E: boolean 플래그

```
// ScheduleSearchRepoImpl.java
private BooleanExpression importantEq(Boolean important) {
    return Boolean.TRUE.equals(important) ? se.important.isTrue() : null;
}
```

4. 응용: 복합 조건 조합

여러 BooleanExpression을 하나로 합치기

```
// 활성화된 과 단위 조직만 조회하는 공통 조건
private BooleanExpression isActiveDepartment() {
    return oe.orgType.in(
```

```

        OrganizationType.DEPARTMENT,
        OrganizationType.HEALTH_SUB_CENTER,
        OrganizationType.MANAGEMENT_CENTER)
        .and(oe.orgActivate.isTrue());
    }

    // 사용
    queryFactory.selectFrom(oe)
        .where(isActiveDepartment())
        .fetch();

```

조건을 변수로 받아서 동적으로 조합

```

// 검색 조건 객체를 받아서 BooleanExpression 배열로 변환
private BooleanExpression[] buildConditions(SearchCondition cond) {
    return new BooleanExpression[] {
        orgNameContains(cond.getOrgName()),
        orgTypeEq(cond.getOrgType()),
        orgActivateEq(cond.getActivate()),
        createdAfter(cond.getStartDate()),
        createdBefore(cond.getEndDate())
    };
}

// 사용
queryFactory.selectFrom(oe)
    .where(buildConditions(condition))
    .fetch();

```

BooleanBuilder와 섞어 쓰기

복잡한 분기 로직이 필요할 때 BooleanBuilder 안에 BooleanExpression을 넣는다.

```

private BooleanBuilder buildFilterCondition(MemberSearchCondition
condition) {
    BooleanBuilder builder = new BooleanBuilder();

    // BooleanExpression 메서드 재사용
    builder.and(orgPublicIdEq(condition.getOrgPublicId()));
    builder.and(positionEq(condition.getMbPosition()));
    builder.and(activateEq(condition.getMbActivate()));

    // 복잡한 분기는 BooleanBuilder에서 처리
    int start = condition.getStartGrade();
    int end = condition.getEndGrade();
    if (start != 0 && end != 0) builder.and(qa.mbGrade.between(start,
end));
    else if (start != 0) builder.and(qa.mbGrade.goe(start));
}

```

```

        else if (end != 0)                builder.and(qa.mbGrade.loe(end));

        return builder;
    }

    // BooleanExpression 메서드들은 재사용 가능
    private BooleanExpression orgPublicIdEq(String orgPublicId) {
        return orgPublicId != null ? qa.publicId.eq(orgPublicId) : null;
    }

    private BooleanExpression positionEq(MemberPositionType position) {
        return position != null ? qa.mbPosition.eq(position) : null;
    }

```

5. 응용: 조건부 OR/AND 전환

```

// 관리자: 비활성 포함 전체 조회 / 일반 사용자: 활성화만 조회
private BooleanExpression activateCondition(boolean isAdmin) {
    return isAdmin ? null : oe.orgActivate.isTrue();
}

queryFactory.selectFrom(oe)
    .where(activateCondition(currentUser.isAdmin()))
    .fetch();
// 관리자 → WHERE 조건 없음 (전체)
// 일반   → WHERE org_activate = true

```

부정 조건 (NOT)

```

// 일반 직급이 아닌 사람만 조회
private BooleanExpression notGeneral() {
    return qa.mbPosition.ne(MemberPositionType.GENERAL);
}

// IN의 반대
private BooleanExpression notTeamLevel() {
    return oe.orgType.notIn(OrganizationType.TEAM,
        OrganizationType.HEALTH_CENTER);
}

```

존재 여부 서브쿼리 (응용)

```

import static com.querydsl.jpa.JPAExpressions.select;

```

```
// 소속 회원이 있는 조직만 조회
private BooleanExpression hasMembers() {
    return oe.id.in(
        select(qi.orgId)
            .from(qi)
            .where(qi.mbActivate.isTrue())
    );
}
```

6. 주의사항

null 반환은 where()에서만 동작한다

```
// where에서 null은 자동 무시
.where(orgNameContains(null)) // 조건 제외됨

// .and() 체이닝에서 null은 NPE 발생
orgNameContains(null).and(orgTypeEq(type)) // NullPointerException!
```

해결: 조합할 때는 null 체크 필요

```
// 안전한 조합 방법
private BooleanExpression combinedCondition(String name, OrganizationType
type) {
    BooleanExpression result = orgNameContains(name);
    BooleanExpression typeCondition = orgTypeEq(type);

    if (result == null) return typeCondition;
    if (typeCondition == null) return result;
    return result.and(typeCondition);
}

// 또는 BooleanBuilder를 사용 (null-safe)
BooleanBuilder builder = new BooleanBuilder();
builder.and(orgNameContains(name)); // null이어도 안전
builder.and(orgTypeEq(type)); // null이어도 안전
```

메서드 네이밍 컨벤션

패턴	네이밍	예시
같은지	필드Eq	orgTypeEq(), positionEq()

패턴	네이밍	예시
포함하는지	필드Contains	orgNameContains() , keywordContains()
이상/이하	필드Goe / 필드Loe	startFromGoe() , endToLoe()
boolean	is필드 / 필드Eq	isActive() , importantEq()
복합 조건	의미 기반	isActiveDepartment() , hasMembers()